

Надёжная обработка заказов

Ошибки, повторы, логи и контроль затрат.

Сработало один раз ≠ можно доверять

Два сценария делают одно и то же

Одинаковый результат на выходе. Разница видна только в плохой день.

v1 · 7 нод

принимает заказ

разбирает моделью

сохраняет в базу

уведомляет менеджера

v2 · 18 нод

принимает заказ

разбирает моделью

сохраняет в базу

уведомляет менеджера

На демонстрации они неразличимы.

Критерии надёжности

Чем «работает» отличается от «можно доверять».

01 переживает отказ внешнего сервиса

02 не делает одну работу дважды

03 при сбое оставляет след, а не тишину

04 тратит предсказуемые деньги

05 останавливается по команде человека

06 восстанавливается по инструкции, а не по памяти автора

Ни один критерий не про функциональность — все про поведение в плохой день.

Шесть способов сломаться

ОТКАЗ

КОГДА СЛУЧАЕТСЯ

Недоступна LLM

сеть, квота, протухший ключ

Превышено время ожидания

модель думает дольше, чем ждёт вызывающий

Пришёл неверный JSON

температура, длинный ответ, markdown-обёртка

Внешний сервис не отвечает

база, мессенджер, CRM

Один webhook пришёл дважды

ретрай на стороне отправителя

Данные записались частично

упало между двумя записями

ТРИ ВОПРОСА К КАЖДОМУ ОТКАЗУ

Что увидит клиент?

Что останется в базе?

Узнает ли кто-нибудь?

На v1 ответ на третий вопрос почти всегда «нет».

Что проверяем до вызова модели

- обязательные поля — `customer` · `contact` · `text`
- типы — `text` строка, `qty` число
- размер входа — не больше 2000 символов
- защита от циклов — заголовок `x-loop-depth`

Каждая проверка здесь
— это несостоявшийся
платный вызов.

Идемпотентность

Результат десяти одинаковых запросов равен результату одного.

ПРИОРИТЕТНЫЙ

Idempotency-Key

прислал отправитель · различает два одинаковых заказа

ЗАПАСНОЙ

Хэш от содержимого

считаем сами · два одинаковых заказа склеит в один

Где стоит проверка дубля

✗ модель → проверка дубля → база

✓ **проверка дубля → модель → база**

Одна нода поиска закрывает две проверки — совпадение ключа и число заказов с контакта за час.

Проверка после вызова модели защищает базу, но не кошелёк.

Безопасный повтор

Что нужно для безопасного перезапуска

- ключ идемпотентности сохранён в строке заказа
- входные данные сохранены в `retry_payload`
- статус отличает «в работе» от «сбой»

ЧЕГО ДЕЛАТЬ НЕЛЬЗЯ

Не жать **Retry** в интерфейсе n8n

Он повторит запуск с середины, а нода создания заказа уже отработала — получите второй заказ.

Запись в два приёма

✓ ДВА ПРИЁМА

create (status: processing) → модель → update (done / failed)

✗ ТОЛЬКО В КОНЦЕ

модель → create (status: done)

Если писать только в конце, упавший заказ исчезает бесследно. Строка со статусом `processing` — это то, что потом можно пометить сбойной и перезапустить.

Таймауты и повторы живут в двух местах

Абстракция агента прячет HTTP-слой — параметры разъезжаются по разным панелям.

ПАРАМЕТР	ГДЕ НАСТРАИВАЕТСЯ	ЗНАЧЕНИЕ
Число повторов	нода Агента → Settings → Retry On Fail	3, пауза 2 с
Таймаут одного вызова	подузел YandexGPT → Options → Timeout	20 с
Таймаут сценария	Workflow Settings	120 с

3 попытки × 20 с = до 60 секунд в худшем случае

Error-ветка

Второй выход агента: при сбое выполнение не падает, а уходит вниз.

- статус failed
- сохранение retry_payload
- алерт человеку

КОД ОШИБКИ

llm_timeout

llm_auth_error

llm_rate_limited

llm_bad_format

llm_unavailable

Разные коды — разные действия. «Что-то пошло не так» не помогает дежурному.

Два уровня страховки

Error-ветка в сценарии

ловит **ожидаемые** сбои

модель, таймаут, формат

настраивается на ноде

Error Workflow на инстансе

ловит **неожиданные**

падение базы, обрыв сети, диск

Settings → Error Workflow

Ветка внутри не поймает падение той ноды, которая эту ветку обслуживает.

Executions — чтение упавшего запуска

- 01 фильтр по статусу Error
- 02 нода, на которой остановилось
- 03 вкладка INPUT упавшей ноды — с какими данными пришли
- 04 вкладка OUTPUT предыдущей — что ей передали

В Settings включить **Save execution progress** — иначе данные упавших нод не сохранятся.

Признак тихой остановки — подозрительно короткое время. Запуск с моделью занимает 1,5–3 с; **Succeeded in 1.1s** значит, что модель не вызывалась.

СЕДЬМОЙ ТИП ОТКАЗА

Отказ, который не выглядит как отказ

Шесть отказов из карты — про падение. Седьмой — про тишину: нода поиска не нашла ничего, не отдала ни одного элемента, цепочка встала.

ЧТО ВИДНО В EXECUTIONS

Succeeded in 1.114s

всё зелёное · ни одной красной ноды ·
результата нет

**Зелёный статус обманывает
сильнее красного.**

Лечится галочкой **Always Output Data** — но сначала
надо догадаться, что искать.

Понятные статусы

СТАТУС	ЧТО ДЕЛАТЬ
processing	ничего, идёт обработка
done	ничего, всё хорошо
needs_review	посмотреть глазами — неизвестный артикул или нет адреса
failed	перезапустить по инструкции
duplicate	ничего, это повтор

Статус — не для базы, а для дежурного: должен читаться как инструкция к действию.

Минимальный журнал

Таблица `order_log` — одна строка на этап.

ts	run_id	order_id	stage	status	duration_ms	tokens	cost_rub	error
----	--------	----------	-------	--------	-------------	--------	----------	-------

validate → **dedup** → **llm** → **parse**

Журнал отвечает «что и когда сломалось» — не «что случилось вообще», иначе станет вторым хранилищем.

Чего в журнале быть не должно

- ✗ текст заказа и любые пользовательские тексты
- ✗ контакт, имя, адрес доставки
- ✗ содержимое ответа модели
- ✗ ключи, токены, значения credentials

Лог с персональными данными — это второе хранилище ПД, к которому применимы те же требования, что и к первому, но про которое все забывают.

Сколько стоит один запуск

$(\text{токены_входа} + \text{токены_выхода}) / 1000 \times \text{цена_за_1000}$

Токены входа $\approx$ длина_текста / 3 для русского + 300 на системный промпт.

est_cost_rub

считается **до** вызова — отсечь дорогой запрос заранее

cost_rub

считается **после** — проверить, что оценка не врёт

Расхождение больше 30% — повод править коэффициент.

Выключатель — три уровня

МЯГКИЙ

строка `orders_enabled = false` в базе

503, понятный отказ

СРЕДНИЙ

тумблер Active в n8n

404, выглядит как поломка

ЖЁСТКИЙ

отзыв API-ключа

всё падает

Мягкий выключатель работает без доступа к n8n — правкой одной ячейки. Важно, когда остановить надо срочно.

Четыре правила

Credentials

только в разделе Credentials. Проверка: выгрузить сценарий в JSON и поискать AQVN — ничего не должно найтись.

Версии

Download перед каждым изменением, файл в репозиторий с датой. История версий в n8n есть только в платных планах.

Инструкция восстановления

одна страница рядом со сценарием.

Учения

раз в месяц перезапустить один заведомо сбойный заказ. Непроверенная инструкция обычно не работает.

Исправленный workflow · журнал проверок · инструкция перезапуска

6 типов отказов закрыто

5 статусов обработки

3 уровня выключателя

0 потерянных заказов при сбое модели

Надёжность стоит фиксированные десять нод. На сценарии из семи это выглядит как рост втрое. На сценарии из сорока — это плюс четверть, и никто не замечает.